

INSTRUCTIVO COMPLETO DE GASTOS HORMIGA

```
```markdown
📖 INSTRUCTIVO: Implementación de Detección de Gastos Hormiga para FinanceBot

Versión: 1.0

Backend: Google Sheets (Producción)

Autor: Vibe Code (Mistral AI)

📌 RESUMEN EJECUTIVO

Este instructivo detalla la implementación completa de la detección de gastos hormiga en FinanceBot, diseñado específicamente para el backend de Google Sheets que se utiliza en producción.

- ✅ Valores por defecto inteligentes
- ✅ Personalización completa por usuario
- ✅ Integración nativa con Google Sheets
- ✅ Detección multicriterio (monto + categoría + frecuencia)
- ✅ Soporte multi-moneda con conversión automática
- ✅ Interfaz de usuario completa

📄 CONTENIDO

1. [Definición y Objetivos](#1-definición-y-objetivos)
2. [Estructura de Google Sheets](#2-estructura-de-google-sheets)
3. [Funciones de Base de Datos](#3-funciones-de-base-de-datos)
4. [Lógica de Detección](#4-lógica-de-detección)
5. [Handlers y Comandos](#5-handlers-y-comandos)
6. [Integración con Menú](#6-integración-con-menú)
7. [Configuración de Comandos](#7-configuración-de-comandos)
8. [Pruebas Recomendadas](#8-pruebas-recomendadas)
9. [Consideraciones Técnicas](#9-consideraciones-técnicas)
10. [Ejemplos de Uso](#10-ejemplos-de-uso)

```

## ## 1. DEFINICIÓN Y OBJETIVOS

### ### ¿Qué son los gastos hormiga?

Pequeños gastos recurrentes que individualmente parecen insignificantes, pero al sumarse representan cantidades considerables:

- Café diario
- Snacks y golosinas
- Transporte corto (taxi, Uber)
- Suscripciones (Netflix, Spotify)
- Comida rápida

### ### Criterios de Detección

Criterio	Valor Default	Configurable	Descripción
----- ----- ----- -----			
<b>**Umbral monetario**</b>	\$5.00 USD	✓	Gastos $\leq$ este monto
<b>**Frecuencia mínima**</b>	3 veces/semana	✓	En misma categoría
<b>**Categorías**</b>	café,snacks,transporte	✓	Detección por keywords
<b>**Moneda**</b>	USD	✓	Moneda base para conversión

---

## ## 2. ESTRUCTURA DE GOOGLE SHEETS

### ### Archivo: `database\_gsheets.py` (Añadir a SHEET\_NAMES y SHEET\_COLUMNS)

```
```python
SHEET_NAMES = {
    # ... existentes ...
    "config_gastos_hormiga": "config_gastos_hormiga",
    "gastos_hormiga": "gastos_hormiga",
}

SHEET_COLUMNS = {
    # ... existentes ...
    "config_gastos_hormiga": [
        "id", "usuario_id", "umbral_base", "umbral_moneda",
        "frecuencia_minima", "categorias_auto", "notificaciones_activas",
        "created_at", "updated_at"
    ],
    "gastos_hormiga": [
```

```

        "id", "transaccion_id", "usuario_id", "categoria",
        "monto", "moneda_id", "fecha", "detectado_en", "es_recorrente"
    ],
}

```

```

DATE_COLUMNS = {
    # ... existentes ...
    "detectado_en", # Para gastos_hormiga
}
...

```

```
---
```

3. FUNCIONES DE BASE DE DATOS

En `database\_gsheets.py` (clase GoogleSheetsDB)

```

```python
def obtener_config_gastos_hormiga(self, usuario_id: int) -> Dict[str, Any]:
 """Obtiene configuración con VALORES POR DEFECTO si no existe."""
 with LOCK:
 configs = self._cache.get("config_gastos_hormiga", [])
 for cfg in configs:
 if int(cfg.get("usuario_id", 0)) == usuario_id:
 return dict(cfg)

VALORES POR DEFECTO
now = self._now()
cfg_id = self._next_id("config_gastos_hormiga")
nueva_config = {
 "id": cfg_id,
 "usuario_id": usuario_id,
 "umbral_base": 5.0, # $5 USD
 "umbral_moneda": "USD",
 "frecuencia_minima": 3, # 3 veces/semana
 "categorias_auto": "café,snacks,transporte,suscripciones,comida rápida",
 "notificaciones_activas": 1, # Activado
 "created_at": now,
 "updated_at": now,
}
configs.append(nueva_config)

```

```

self._cache_dirty.add("config_gastos_hormiga")
self._schedule_flush()
return dict(nueva_config)

def guardar_config_gastos_hormiga(self, usuario_id: int, config: Dict[str, Any]) -> Dict[str, Any]:
 """Guarda configuración PERSONALIZADA."""
 with LOCK:
 configs = self._cache.get("config_gastos_hormiga", [])
 existente = next((c for c in configs if int(c.get("usuario_id", 0)) == usuario_id), None)
 now = self._now()

 if existente:
 existente.update({
 "umbral_base": config.get("umbral_base", existente.get("umbral_base", 5.0)),
 "umbral_moneda": config.get("umbral_moneda", existente.get("umbral_moneda",
"USD")),
 "frecuencia_minima": config.get("frecuencia_minima",
existente.get("frecuencia_minima", 3)),
 "categorias_auto": config.get("categorias_auto", existente.get("categorias_auto", "")),
 "notificaciones_activas": config.get("notificaciones_activas",
existente.get("notificaciones_activas", 1)),
 "updated_at": now,
 })
 else:
 cfg_id = self._next_id("config_gastos_hormiga")
 nueva_config = {
 "id": cfg_id, "usuario_id": usuario_id,
 "umbral_base": config.get("umbral_base", 5.0),
 "umbral_moneda": config.get("umbral_moneda", "USD"),
 "frecuencia_minima": config.get("frecuencia_minima", 3),
 "categorias_auto": config.get("categorias_auto",
"café,snacks,transporte,suscripciones"),
 "notificaciones_activas": config.get("notificaciones_activas", 1),
 "created_at": now, "updated_at": now,
 }
 configs.append(nueva_config)

 self._cache_dirty.add("config_gastos_hormiga")
 self._schedule_flush()
 return self.obtener_config_gastos_hormiga(usuario_id)

```

```

def registrar_gasto_hormiga(self, transaccion_id: int, usuario_id: int, categoria: str,
 monto: float, moneda_id: Optional[int] = None) -> Dict[str, Any]:
 """Registra un gasto como hormiga."""
 with LOCK:
 gastos = self._cache.get("gastos_hormiga", [])
 now = self._now()
 gh_id = self._next_id("gastos_hormiga")
 nuevo = {
 "id": gh_id, "transaccion_id": transaccion_id, "usuario_id": usuario_id,
 "categoria": categoria, "monto": round(float(monto), 2),
 "moneda_id": moneda_id if moneda_id else "", "fecha": now[:10],
 "detectado_en": now, "es_recorrente": 1,
 }
 gastos.append(nuevo)
 self._cache_dirty.add("gastos_hormiga")
 self._schedule_flush()
 return dict(nuevo)

def obtener_gastos_hormiga(self, usuario_id: int, dias: int = 30) -> List[Dict[str, Any]]:
 """Obtiene gastos hormiga con JOIN manual a transacciones y monedas."""
 gastos = self._cache.get("gastos_hormiga", [])
 transacciones = self._cache.get("transacciones", [])
 monedas = self._cache.get("monedas", [])

 txn_lookup = {int(t.get("id", 0)): t for t in transacciones}
 moneda_lookup = {int(m.get("id", 0)): m for m in monedas}
 resultado = []
 hoy = datetime.now()

 for g in gastos:
 if int(g.get("usuario_id", 0)) != usuario_id:
 continue
 fecha_g = g.get("fecha", "")
 if fecha_g:
 try:
 if (hoy - datetime.strptime(fecha_g[:10], "%Y-%m-%d")).days > dias:
 continue
 except ValueError:
 continue

 row = dict(g)

```

```

txn = txn_lookup.get(int(g.get("transaccion_id", 0)))
if txn:
 row.update({
 "transaccion_descripcion": txn.get("descripcion", ""),
 "transaccion_fecha": txn.get("fecha", ""),
 })

```

```

mid = g.get("moneda_id")
if mid and mid in moneda_lookup:
 m = moneda_lookup[mid]
 row.update({
 "moneda_simbolo": m.get("simbolo", "$"),
 "moneda_abreviatura": m.get("abreviatura", ""),
 })

```

```

resultado.append(row)

```

```

resultadoxsort(key=lambda x: str(x.get("detectado_en", "")), reverse=True)
return resultado

```

```

def obtener_estadisticas_gastos_hormiga(self, usuario_id: int, dias: int = 30) -> Dict[str, Any]:
 """Estadísticas agregadas por categoría."""
 gastos = self.obtener_gastos_hormiga(usuario_id, dias)
 por_categoria = {}
 total = 0.0
 cantidad = 0

 for g in gastos:
 cat = g.get("categoria", "Otros")
 monto = float(g.get("monto", 0))
 if cat not in por_categoria:
 por_categoria[cat] = {"total": 0.0, "cantidad": 0}
 por_categoria[cat]["total"] += monto
 por_categoria[cat]["cantidad"] += 1
 total += monto
 cantidad += 1

 return {
 "total": round(total, 2),
 "cantidad": cantidad,
 "por_categoria": [{"categoria": k, **v} for k, v in sorted(por_categoria.items(), key=lambda

```

```
x: x[1]["total"], reverse=True)],
 }
 ...

```

```

```

## ## 4. LÓGICA DE DETECCIÓN

```
En `knowledge.py` (añadir al inicio)

```

```
`python

```

```
Categorías y factores de conversión

```

```
CATEGORIAS_HORMIGA_DEFAULT = {

```

```
 "café": ["café", "cafe", "starbucks", "coffee", "capuchino"],
 "snacks": ["snack", "galleta", "chocolate", "dulce", "golosina"],
 "transporte": ["taxi", "uber", "didi", "cabify", "bici", "moto", "bus"],
 "suscripciones": ["netflix", "spotify", "youtube premium", "amazon prime"],
 "comida rápida": ["mcdonalds", "burger king", "pizza", "hamburguesa", "delivery"],
}
```

```
FACTORES_CONVERSION = {

```

```
 "USD": 1.0, "EUR": 1.08, "CUP": 0.0417, "MLC": 1.0,
 "MXN": 0.058, "COP": 0.00025, "ARS": 0.0011, "CLP": 0.0011,
}
```

```
def detectar_gasto_hormiga(usuario: Dict[str, Any], monto: float,

```

```
 descripcion: str, categoria: str,

```

```
 moneda: Optional[Dict[str, Any]] = None) -> bool:

```

```
 """Detecta si una transacción es gasto hormiga."""

```

```
 try:

```

```
 config = database.obtener_config_gastos_hormiga(usuario["id"])

```

```
 # 1. Verificar por monto

```

```
 umbral = config.get("umbral_base", 5.0)

```

```
 umbral_moneda = config.get("umbral_moneda", "USD").upper()

```

```
 moneda_abrev = (moneda.get("abreviatura", "USD") or "USD").upper() if moneda else

```

```
 "USD"

```

```
 if moneda_abrev != umbral_moneda:

```

```
 factor_moneda = FACTORES_CONVERSION.get(moneda_abrev, 1.0)

```

```
 factor_umbral = FACTORES_CONVERSION.get(umbral_moneda, 1.0)

```

```

 if factor_moneda > 0 and factor_umbral > 0:
 umbral = (umbral * factor_umbral) / factor_moneda

 if monto > umbral:
 return False

 # 2. Verificar por categoría
 categorias_activas = [c.strip().lower() for c in config.get("categorias_auto", "").split(",") if
c.strip()]
 if categoria and categoria.lower() in categorias_activas:
 return True

 descripcion_lower = (descripcion or "").lower()
 for cat, keywords in CATEGORIAS_HORMIGA_DEFAULT.items():
 if cat.lower() not in categorias_activas:
 continue
 if any(kw in descripcion_lower for kw in keywords):
 return True

 # 3. Verificar por frecuencia (opcional, más costoso)
 return _es_gasto_hormiga_por_frecuencia(usuario["id"], categoria, config)

except Exception as e:
 logger.error("Error detectando gasto hormiga: %s", e)
 return False

def _es_gasto_hormiga_por_frecuencia(usuario_id: int, categoria: str, config: Dict[str, Any]) ->
bool:
 """Verifica frecuencia de gastos en categoría."""
 from datetime import datetime, timedelta
 frecuencia_minima = config.get("frecuencia_minima", 3)
 transacciones = database.obtener_transacciones(usuario_id, limite=100)
 hoy = datetime.now()
 count = 0

 for t in transacciones:
 fecha_str = t.get("fecha", "")[:10]
 if not fecha_str:
 continue
 try:
 fecha_t = datetime.strptime(fecha_str, "%Y-%m-%d")

```

```

 if (hoy - fecha_t).days > 7: # Última semana
 continue
 except ValueError:
 continue

 cat_trans = (txget("categoria_nombre") or "").lower()
 desc_trans = (txget("descripcion") or "").lower()
 if categoria.lower() in cat_trans or categoria.lower() in desc_trans:
 count += 1

 return count >= frecuencia_minima
'''

```

---

## ## 5. HANDLERS Y COMANDOS

### En `handlers.py`

```

'''python
async def consultar_gastos_hormiga(update: Update, context: ContextTypes.DEFAULT_TYPE):
 """Comando /gastos_hormiga"""
 try:
 usuario = _obtener_usuario_contexto(update, context)
 texto = knowledge._procesar_gastos_hormiga(usuario)
 await update.message.reply_text(texto, parse_mode="Markdown",
reply_markup=_crear_teclado_principal())
 except Exception as e:
 logger.error("Error en /gastos_hormiga: %s", e)
 await update.message.reply_text("⚠ Error al obtener gastos hormiga.",
reply_markup=_crear_teclado_principal())

async def configurar_gastos_hormiga(update: Update, context: ContextTypes.DEFAULT_TYPE):
 """Comando /config_hormiga"""
 try:
 usuario = _obtener_usuario_contexto(update, context)
 args = context.args or []
 mensaje = " ".join(args) if args else "mostrar"
 texto = knowledge._procesar_config_gastos_hormiga(usuario, mensaje)
 await update.message.reply_text(texto, parse_mode="Markdown",
reply_markup=_crear_teclado_principal())

```

```

except Exception as e:
 logger.error("Error en /config_hormiga: %s", e)
 await update.message.reply_text("⚠ Error al procesar configuración.",
reply_markup=_crear_teclado_principal())
...

```

---

## ## 6. INTEGRACIÓN CON MENÚ

### En `menus.py` (teclado\_principal)

```

```python
def teclado_principal() -> InlineKeyboardMarkup:
    botones = [
        [InlineKeyboardButton("💰 Balance", callback_data="accion_balance"),
         InlineKeyboardButton("🇮🇹 Resumen", callback_data="menu_resumen")],
        [InlineKeyboardButton("💵 Gastos", callback_data="accion_gastos"),
         InlineKeyboardButton("💳 Ingresos", callback_data="accion_ingresos")],
        [InlineKeyboardButton("📅 17 Presupuestos", callback_data="menu_presupuestos"),
         InlineKeyboardButton("🎯 Metas", callback_data="menu_metas")],
        # NUEVO:
        [InlineKeyboardButton("🐜 Gastos Hormiga", callback_data="menu_gastos_hormiga"),
         InlineKeyboardButton("⚙ Configurar", callback_data="menu_configurar")],
        [InlineKeyboardButton("📤 Exportar", callback_data="menu_exportar"),
         InlineKeyboardButton("🔄 Más opciones", callback_data="menu_mas")],
    ]
    return InlineKeyboardMarkup(botones)
...

```

En `handlers.py` (callback)

```

```python
elif query.data == "menu_gastos_hormiga":
 texto = knowledge._procesar_gastos_hormiga(usuario)
 await context.bot.send_message(
 chat_id=query.message.chat_id,
 text=texto,
 parse_mode="Markdown",
 reply_markup=_crear_teclado_principal(),
)

```

```

7. CONFIGURACIÓN DE COMANDOS

En `main.py`

```
```python
COMANDOS_MENU = [
 # ... existentes ...
 BotCommand("gastos_hormiga", "Ver tus gastos hormiga (pequeños y recurrentes)"),
 BotCommand("config_hormiga", "Configurar detección de gastos hormiga"),
 # ... restantes ...
]
```

```
En _build_app():
app.add_handler(CommandHandler("gastos_hormiga", consultar_gastos_hormiga))
app.add_handler(CommandHandler("config_hormiga", configurar_gastos_hormiga))
```
```

8. PRUEBAS RECOMENDADAS

| Prueba | Comando | Resultado Esperado |
|-----------------------|---------------------------------|-------------------------------|
| ----- | ----- | ----- |
| Configuración default | `/config_hormiga` | Muestra valores por defecto |
| Detección básica | `Gasté \$3 en café` | Detecta como gasto hormiga |
| Umbral personalizado | `/config_hormiga umbral 10 EUR` | Actualiza umbral a €10 |
| Frecuencia | 4 gastos de café en semana | Todos detectados como hormiga |
| Reporte | `/gastos_hormiga` | Muestra estadísticas y lista |

9. CONSIDERACIONES TÉCNICAS

- ✅ **Google Sheets**: Hojas se crean automáticamente con `\_ensure\_sheets()`
- ✅ **Thread Safety**: Todas las funciones usan `with LOCK`:
- ✅ **Cache**: Integración con sistema de cache/flush existente
- ✅ **Multi-moneda**: Conversión automática con factores estáticos

✅ ****Error Handling****: Manejo de casos edge (usuario sin config, monedas desconocidas)

10. EJEMPLOS DE USO

Configuración:

...

Usuario: /config_hormiga umbral 250 CUP

Bot: ✅ Umbral actualizado a ****250.00 CUP****

Usuario: /config_hormiga categorías café,taxi,netflix

Bot: ✅ Categorías automáticas actualizadas: ****café,taxi,netflix****

...

Detección:

...

Usuario: Gasté \$3 en café

Bot: 🐜 Gasto registrado: ****\$3.00**** en ****café****

💡 ¡Gasto hormiga detectado! Este pequeño gasto puede sumar mucho.

...

Reporte:

...

Usuario: /gastos_hormiga

Bot: 🐜 Tus gastos hormiga (últimos 30 días)

💰 Total: \$45.20 | 📋 15 transacciones

📊 Por categoría:

- café: \$22.50 (8)
- transporte: \$15.00 (5)

💡 Sugerencia: Podrías ahorrar ****\$36.16/mes**** reduciendo un 80%.

...

📁 ARCHIVOS A MODIFICAR

...

FinanceBot/

|— main.py # ✅ Comandos y handlers

```
|—— handlers.py      # ✅ Funciones de comandos
|—— knowledge.py     # ✅ Lógica de detección
|—— database_gsheets.py # ✅ ⭐ Tablas + CRUD
|—— menus.py        # ✅ Botón en menú
|—— formato.py      # ✅ Emoji (opcional)
```

...

🎯 **RESUMEN PARA OPENCODE**

****Objetivo:**** Implementar detección de gastos hormiga en FinanceBot con backend Google Sheets.

****Requisitos:****

1. Crear hojas `config\_gastos\_hormiga` y `gastos\_hormiga` en Google Sheets
2. Implementar funciones CRUD en `database\_gsheets.py` (con valores por defecto)
3. Añadir lógica de detección en `knowledge.py` (multi-moneda + categorías configurables)
4. Añadir comandos `/gastos\_hormiga` y `/config\_hormiga` en `handlers.py` y `main.py`
5. Integrar botón en menú principal (`menus.py`)

****Valores por defecto:****

- Umbral: \$5.00 USD
- Frecuencia: 3 veces/semana
- Categorías: café,snacks,transporte,suscripciones,comida rápida
- Notificaciones: Activadas
